



Macromedia University of Applied Sciences

Course Title: Software-Project 1

Name of Examiner: Prof. Aykut Bußian, Ahmet Yildiz

To be completed by students:

302560, 304190, 304415

Student ID number

B-UBr DT ATI 6e 24S, B-Ubr DT SWE 6e 24S, B-Ubr DT ATI 6e 24W

Matriculation

Gautam, B K , Antony

Last name

Jayant, Ajaya, Nixon

First name

The student work will be submitted as:

(Please fill in the letter X in the appropriate box)

☐

Individual work

☒

Group work

Does only apply to group work: (Complete only if it is a group work)

If you submit a group assignment, please list the first and last names of all group members. By entering their names, the students confirm that they agree to the assignment being submitted in its current form. The contribution of each group member must be indicated in the assignment (e.g. in the outline or chapter headings). Furthermore, by entering their name, the student declares that the entire project work, and in particular the part created by each group member, has been produced independently and without outside help. No aids other than those listed in the attached list of sources and AI tools have been used. All passages taken verbatim or paraphrased from publications are identified as such. Content generated using AI has been marked at the relevant point. Furthermore, it is confirmed that the use of AI tools and AI-supported aids is listed in full in the attached AI directory. It is also assured that all AI-generated content has been checked to the best of our knowledge and belief and in accordance with the general principles of good scientific practice. The work has not yet been submitted to any examination authority in the same or a similar form. By submitting the work, the group members agree that all assessments and comments made by the examiners will be stored in the uploaded work. The group member who uploaded the work must make the correction notes available to the other group members.

1) Jayant Gautam

2) Ajaya B K

3) Nixon Antony

4)

5)

6)

7)

8)

Assessment of group work:

(Please fill in the letter X in the appropriate box)

☐

I apply for an individual evaluation (i.e. each member of the group will receive an individual mark)

☒

I apply for a group evaluation (i.e. each member of the group receives an identical grade)

Berlin, 22.01.2026

Place/Date

Jayant Gautam, Ajaya B K, Nixon

Antony

Complete First Name and Last Name

Evaluation (according to grading scale), result of initial inspection: total points: _____

Date:

Name, first name First Examiner (to be filled in digitally)

To be completed by the examiner: (Text area for the second examiner)

SafeSpend: Fraud Risk Dashboard

Project Title

SafeSpend: Fraud Risk Dashboard

Team Members & Roles

- **Jayant Gautam** – Frontend Developer
- **Ajaya BK** – Database & Backend Developer
- **Nixon Antony** – Tester

1. Abstract

SafeSpend is a full-stack Fraud Risk Dashboard developed to help monitor and analyze financial transactions and identify potentially fraudulent activity. The system provides an interactive dashboard interface where users can view transaction details, track fraud risk levels, visualize patterns using charts, and export fraud reports in PDF and CSV formats. SafeSpend improves fraud investigation efficiency by presenting transaction insights in a structured and user-friendly manner.

2. Problem Statement

Fraudulent transactions cause major financial losses and reduce trust in digital payment systems. Manual fraud monitoring is time-consuming and may miss important patterns. A dashboard-based system is required to analyze transactions efficiently, detect suspicious behavior, and support reporting for daily checkups and also for auditing and compliance.

3. Objective

- To build a dashboard for monitoring transaction data.
- To classify and identify suspicious transactions using fraud risk ml logic.
- To visualize fraud-related trends and insights.
- To export fraud reports in **PDF** and **CSV** formats.
- To provide a simple interface for fraud analysis with graphs and charts and decision-making.

4. System Overview

SafeSpend allows users to view transactions stored in a database and automatically assigns a fraud risk category. The dashboard displays fraud analytics and allows exporting reports for investigation and compliance.

Workflow

1. Transaction data(CSV) is stored in the database.
2. Backend APIs fetch and process the transactions.
3. Fraud scoring logic evaluates transaction risk.
4. Frontend dashboard displays results + charts.
5. Reports can be exported as PDF/CSV.

5. System Architecture

SafeSpend follows a client-server architecture:

Frontend

-Dashboard UI using **HTML, CSS, JavaScript**.

-Displays transaction chart, table and fraud analytics

Backend

-Built using **Flask (Python)**

-Provides REST APIs for transactions, summary, exports, etc.

Database

-**SQLite** database stores transaction records

Report Generation

-Export fraud report as:

CSV
PDF

6. Modules Description

Module 1: Transaction Management

- Stores transaction records in SQLite.
- Retrieves transactions through backend APIs.
- Displays transactions in a structured dashboard table.

Module 2: Fraud Risk Scoring / Detection

- Risk scoring logic integrated in backend.
- Transactions categorized as:

Low Risk

Medium Risk

High Risk

Module 3: Dashboard Analytics & Visualization

- Charts show fraud distribution/trends.
- Helps identify suspicious patterns quickly.

Module 4: Export Reports

- Export suspicious transactions in CSV.
- Export fraud summary/report in PDF.
- Useful for auditing and investigation.

7. Tools & Technologies Used

Frontend

- HTML
- CSS
- JavaScript
- Chart rendering (chart.js)

Backend

- Python
- Flask
- REST APIs

Database

- SQLite

Reports

- CSV Export
- PDF Export

8. Team Contribution

8.1 Jayant Gautam – Frontend Developer

Role Summary

I Worked as the **Frontend Developer** for the SafeSpend project. The main responsibility was to design and build the **Fraud Risk Dashboard interface** and ensure it properly displays transaction data, fraud insights, and user actions like exporting reports focusing on making the dashboard clean, interactive, and easy to use.

Work Done

1) Designing and Building the Dashboard UI

Create an overall dashboard layout using **HTML** ([index.html](#)). Structure the UI in a way that makes fraud monitoring easy by separating it into sections such as:

- Dashboard heading / Main title area
- Fraud analytics section (charts/visuals)
- Transaction display section (table view)
- Action buttons like upload, clear, and export.

This helped the dashboard look organized and made navigation simple for the user.

2) Styling and Improving UI Appearance

CSS (`styles.css`) is used to style the dashboard and ensure the interface looks professional. Working on:

- Proper spacing and alignment of components
- Clean table formatting for transaction readability
- Consistent button styles and icon usage
- Making risk levels visually clear (Low/Medium/High)

Also, use the icons provided in `assets/icons/` to improve the user experience and make actions easy to identify.

3) Enhancing Visual Features using Charts (Improvement Phase)

In the initial stage, the development of frontend was without using any chart dependencies. At that point, the dashboard was functional but the charts looked very basic.

To improve the user experience and make fraud insights easier to understand, I decided to integrate a chart dependency (**`Chart.js`**) into the project. This enhancement helped in:

- Making the dashboard more visually appealing
- Representing fraud risk information clearly through good animations of graphs

This upgrade significantly improved the overall look and effectiveness of the dashboard.

4) Making the Dashboard Dynamic Using JavaScript

Implement the main frontend functionality in **JavaScript (`script.js`)**. Instead of showing static data, I connected the dashboard to backend APIs so that it fetches and updates data automatically.

The frontend logic includes:

- Calling backend APIs to fetch transaction data

- Displaying transactions dynamically inside the table
- Updating summary details whenever data changes
- Ensuring smooth interaction between user actions and backend responses

5) Fraud Analytics and Chart Integration

After integrating **Chart.js**, I connected the fraud analytics section (**chart.js**) with backend summary data. The charts help represent fraud-related insights such as:

- Distribution of transactions based on risk category (Low/Medium/High)
- Fraud trend insights (based on available summary data)

This made the dashboard more interactive and improved decision-making for the user.

6) Implementing Action Buttons (Upload / Clear / Export)

Work on the frontend interface for key dashboard actions:

Upload: Connected the upload UI to the backend upload API so new data can be processed

Clear: Added a clear/reset option that removes transactions and refreshes the UI

Export: Integrated export buttons that allow downloading fraud reports in:

CSV format

PDF format

7) Basic Error Handling and User Experience Improvements

To make the system stable, ensuring the dashboard behaves properly in different cases such as:

- When there is no data to display
- When the backend is not responding
- When export is triggered and the UI needs to refresh

This improved the overall usability and reliability of the frontend.

Files worked on

1) index.html — Dashboard Structure (UI Skeleton)

What it contains:

This is the **main page** of SafeSpend.

Jayant used this file to create the **layout and structure** of the dashboard.

What it do here:

- Created the dashboard sections like:
 - Page header / title (SafeSpend)
 - Buttons (Upload, Export, Clear, etc.)
 - Transaction table section
 - Chart section
 - Summary / metric cards (if present)
- Added placeholders/divs for:
 - Where transaction data will appear dynamically
 - Where charts will be displayed

Why it matters:

Without `index.html`, there is no UI layout to display anything.

It is basically the framework/skeleton of the dashboard.

2) styles.css — Dashboard Styling (Look & Feel)

What it contains:

This file controls the **design and appearance** of the dashboard.

What Jayant did here:

- Made the dashboard look professional by adding:
 - Proper spacing, margins, padding
 - Alignment of sections using flex/grid
 - Button styling (hover effects, size, consistent design)
 - Table styling (row spacing, headers, readable format)
- Ensured risk levels look visually different:
 - Low / Medium / High risk are easier to identify

- Used icons properly by controlling:
 - Icon size
 - Positioning next to buttons/text
 - UI consistency

Why it matters:

Even if the dashboard works technically, if it looks plain/basic, it won't feel like a real product. CSS makes SafeSpend **user-friendly and presentable**.

3) script.js — Dashboard Functionality + API Connection (Brain of Frontend)

What it contains:

This file is responsible for **making the dashboard interactive**.

What Jayant did here:

- Connected frontend to backend APIs such as:
 - Fetching transactions from backend
 - Loading summary stats
 - Refreshing the table after upload/clear
- Implemented dynamic table rendering:
 - Instead of hardcoding rows in HTML, rows are created using JS
 - Added button event handlers:
 - When user clicks export → call backend export endpoint
 - When user clicks clear → clear data and refresh UI
 - When user uploads → send file/data to backend and update UI

Why it matters:

Without `script.js`, the dashboard would be static (no real-time updates, no backend communication).

This file makes SafeSpend feel like a real working application.

4) chart.js — Visual Analytics (Fraud Charts)

What it contains:

This file handles the **charts and visual representation** of fraud data.

What Jayant did here :

Initially, built the dashboard without any chart dependency, so the dashboard was functional but looked basic.

Later, to improve visuals and user understanding, he integrated Chart.js and used `chart.js` to:

- Display fraud distribution visually
- Show risk-level breakdown in graphs
- Make fraud patterns easier to understand quickly

Why it matters:

Charts improve the dashboard by making it:

- More professional
- More visually attractive
- Easier for users to analyze fraud trends

Final Outcome

By completing the frontend work, I delivered a fully functional and user-friendly **SafeSpend Fraud Risk Dashboard interface** that:

- Displays transaction data dynamically
- Provides enhanced fraud insights using charts
- Allows users to export reports in PDF/CSV
- Looks professional and is easy to navigate

8.2 Ajaya BK – Backend Developer

Role Summary

Working as the **Backend Developer** for the SafeSpend project. The responsibility was to build the complete backend system that powers the Fraud Risk Dashboard. This included developing the Flask server, creating REST APIs, managing the database, integrating fraud scoring logic, and implementing export features for generating fraud reports in PDF and CSV formats.

Work Done

1) Backend Development using Flask

Develop the backend of SafeSpend using **Flask (Python)**. The backend acts as the core processing unit of the system and handles all operations such as transaction retrieval, summary generation, fraud risk evaluation, and report exports.

Ensured that the backend provides smooth communication between:

- The frontend dashboard
- The database
- The fraud detection/risk scoring logic
- The report export module

2) API Design and Implementation

Create REST API endpoints so the frontend can fetch and display data dynamically. These APIs are responsible for returning transaction details, generating summaries, and enabling export operations.

Some key API endpoints include:

-Fetch all transactions

`GET /api/transactions`

Used to load the transaction list on the dashboard.

-Fetch a specific transaction

`GET /api/transactions/<int:transaction_id>`

Used to view a single transaction in detail.

-Get transaction summary

`GET /api/transactions/summary`

Used to display dashboard metrics such as total transactions and risk-level distribution.

-Upload transactions

`POST /api/upload` Used to upload new transaction data into the system.

-Clear all transactions

`POST /api/clear`

Used to reset the database transaction records and refresh the dashboard.

-Export reports

`GET /api/export/csv`

`GET /api/export/pdf`

Used to generate and download fraud reports.

These endpoints made the dashboard fully functional and allowed real-time updates between the frontend and backend.

3) Database Integration using SQLite

Connect the backend to a **SQLite database** to store and manage transaction records. This allowed SafeSpend to persist transaction information and fetch it efficiently when required.

Worked on:

- Defining database schema/models
- Performing CRUD operations such as:
 - inserting uploaded transaction records
 - retrieving transaction history
 - clearing records when requested

- Ensuring database operations are consistent and stable

This database integration ensures SafeSpend can work as a complete transaction monitoring system rather than a temporary/demo application.

4) Fraud Risk Scoring / ML Integration

Integrate the fraud scoring logic through the backend using `ml_integration.py`.

This module is responsible for analyzing transaction details and assigning a fraud risk category such as:

Low Risk

Medium Risk

High Risk

This feature is important because it helps the dashboard highlight suspicious transactions and assists the user in identifying fraud patterns faster.

5) Report Generation and Export Feature (PDF & CSV)

To support fraud investigation and auditing, I implemented report export functionality using `export_utils.py`.

The export module generates:

-CSV Report for data analysis and record keeping

-PDF Report for documentation and presentation

Reports are saved inside the project under:

`exports/`

This feature makes SafeSpend more practical, because users can generate evidence and reports for compliance purposes.

6) Backend Testing and Debugging Support

Ensure that backend responses were accurate and properly formatted for frontend usage and also performed backend testing by verifying:

- Correctness of API responses
- Proper handling of empty data states
- Stability of export feature generation
- Smooth connection between frontend actions and backend processing

This helped maintain a reliable backend and reduced system errors.

Technologies Used

Python

Flask

SQLite

REST API development

Fraud scoring integration

PDF and CSV export utilities

Files I Worked On

1) app.py — Main Flask Server + API Controller (Backend Core)

What this file is:

`app.py` is the **main backend entry point** of the SafeSpend project.

It starts the Flask server and controls how the backend responds to frontend requests.

What Ajaya did in this file:

Ajaya implemented the main backend logic such as:

Flask app setup

- Configured the Flask application
- Enabled CORS so the frontend can communicate with backend smoothly

API route creation

He created endpoints that the frontend calls to perform actions like:

- Fetch transactions
- View transaction summary
- Upload data
- Clear data
- Export reports

Example endpoints from the project:

```
GET /api/transactions
```

```
GET /api/transactions/<transaction_id>
```

```
GET /api/transactions/summary
```

```
POST /api/upload
```

```
POST /api/clear
```

```
GET /api/export/csv
```

```
GET /api/export/pdf
```

Request handling

- Read inputs sent from the frontend (like upload requests)
- Returned clean JSON responses to the dashboard

Response management

- Ensured data is returned in correct format for frontend usage
- Handled success/failure responses properly

Why it matters:

Without `app.py`, there would be **no backend running**, and the frontend dashboard would not be able to load real transaction data or export reports.

2) models.py — Database Models + Transaction Storage Logic

What this file is:

`models.py` defines the **database structure** and handles how transactions are stored and retrieved from the SQLite database.

This is basically the **data layer** of SafeSpend.

What Ajaya did in this file:

Ajaya worked on:

Database schema design

- Defined how a transaction record should look in the database
- Includes fields like:

- transaction ID
- amount
- merchant/user info
- timestamp
- risk score / risk label (if stored)

Database operations

- Logic to insert transaction data
- Logic to query transactions
- Logic to clear/delete transactions when reset is requested

Why it matters:

Even if the dashboard UI is perfect, without a database model:

- Data cannot be stored permanently
- Transactions cannot be fetched reliably
- Summary calculations become inaccurate

So `models.py` is essential for making SafeSpend a **real system**, not just a demo.

3) `ml_integration.py` — Fraud Risk Scoring / Classification Logic

What this file is:

`ml_integration.py` is responsible for the **fraud detection intelligence part** of SafeSpend.

Even if it's rule-based or ML-based, this file is the module that decides:

- “Is this transaction suspicious?”
- “How risky is this transaction?”

What Ajaya did in this file:

Ajaya integrated fraud detection logic such as:

Risk score generation

- Computes fraud risk score based on transaction features

Risk level classification

- Converts risk score into categories like:

- Low Risk
- Medium Risk

-High Risk

Backend integration

-Ensured that when transactions are fetched or uploaded, they get processed through fraud scoring logic before being shown on the dashboard.

Why it matters:

This file is the **main purpose of SafeSpend**.

Without this module, SafeSpend would only be a “transaction viewer” — but with it, SafeSpend becomes a **Fraud Risk Dashboard**.

4) export_utils.py — Report Generation (PDF + CSV)

What this file is:

`export_utils.py` is the module responsible for creating **downloadable fraud reports**.

It converts transaction and fraud data into:

- CSV report** (for spreadsheets / analysis)
- PDF report** (for documentation / submission / audit)

What Ajaya did in this file:

Ajaya implemented:

CSV Export logic

- Formats transaction data into rows/columns
- Saves output into `exports/` folder

PDF Export logic

- Generates a readable fraud report document
- Includes important fraud information like transaction details and risk category

Backend connection

-Connected export functions to API routes:

```
GET /api/export/csv  
GET /api/export/pdf
```

Why it matters:

Exporting is an important “real-world feature” because fraud teams need reports for:

- audit purposes
- investigation evidence
- sharing results with authorities or managers

So this module makes SafeSpend more practical and complete.

5) instance/transactions.db — SQLite Database File (Stored Data)

What this file is:

This is the actual **SQLite database file** where transaction records are stored.

It contains the saved data used by the backend.

What Ajaya did with this file:

Ajaya ensured that:

- The backend connects properly to this database
- Transactions are inserted and retrieved correctly
- Data persists even after restarting the server
- Clearing/resetting removes records correctly

Why it matters:

This file proves SafeSpend supports **real stored data**, not temporary runtime data.

It enables:

- history tracking
- consistent fraud analysis
- reliable dashboard loading

Final Outcome

By implementing the backend, I ensured SafeSpend works as a complete fraud monitoring system. The backend successfully:

- Stores and retrieves transactions from the database
- Provides APIs for the frontend dashboard
- Assigns fraud risk levels to transactions
- Generates and exports fraud reports in PDF and CSV formats

This backend system forms the foundation of SafeSpend and enables the dashboard to function efficiently.

8.3 Nixon Antony – Tester

Role Summary

Work as the **Tester** for the SafeSpend project. The main responsibility was to verify that the Fraud Risk Dashboard works correctly from the user's perspective and that all features perform as expected. Testing the frontend interface, validated backend API responses, checked fraud risk outputs, and ensured that report exports (PDF and CSV) were generated properly.

Work Done

1) Functional Testing of Dashboard Features

Test the major features of the SafeSpend dashboard to confirm correct behavior, including:

- Loading transaction data into the dashboard table
- Checking that transactions are displayed with correct values (amount, merchant, timestamp, etc.)
- Verifying that risk labels (Low/Medium/High) are shown correctly
- Confirming that summary data updates properly when transactions change

This ensured the dashboard was usable and reliable for fraud analysis.

2) API Testing and Validation

Since SafeSpend uses Flask APIs to fetch and manage data, Verify that the backend endpoints work correctly by checking:

- Correct status codes (success/failure responses)
- Correct JSON output format for transaction and summary APIs
- Data consistency between backend database and frontend display
- Behavior when the database is empty or cleared

This ensured smooth communication between frontend and backend.

3) Report Export Testing (PDF & CSV)

Test the report generation and export features to ensure they functioned properly, including:

- Exporting fraud reports as **CSV**
- Exporting fraud reports as **PDF**
- Confirming that exported files contain valid transaction data
- Verifying that exported files are properly created and stored inside the `exports/` folder
- Checking for formatting issues and missing values in the report

This step was important for auditing and documentation reliability.

4) UI Testing and User Experience Checks

Test the UI from an end-user perspective to ensure the dashboard looks correct and is easy to use. This included:

- Checking layout alignment and spacing
- Verifying button usability and icon clarity
- Ensuring the dashboard remains readable across different screen sizes
- Testing navigation flow and ease of understanding

This helped improve the overall quality and presentation of the dashboard.

5) Edge Case and Error Handling Testing

Test how the system behaves in special conditions such as:

- No transactions available (empty dataset)
- Uploading invalid/unsupported data formats
- Backend server not responding
- Export triggered when there is no data
- Refreshing after clearing the database

This helped ensure the system remains stable and does not crash under unexpected conditions.

6) Bug Reporting and Fix Verification

When issues arise, Document them and share them with the team. After fixes were applied, re-testing the same features to confirm:

- The bug was resolved
- No new issues were introduced
- The system remained stable after updates

This improved the reliability of the final project.

Testing Types Covered

- Functional Testing** (features work correctly)
- UI Testing** (layout and usability)
- API Testing** (response correctness and stability)
- Report Testing** (PDF/CSV export validation)
- Edge Case Testing** (empty data, invalid inputs)

Tools / Methods Used

- Manual testing through the dashboard UI
- Checking API outputs through browser/testing tools
- Validating exported files by opening and reviewing output content

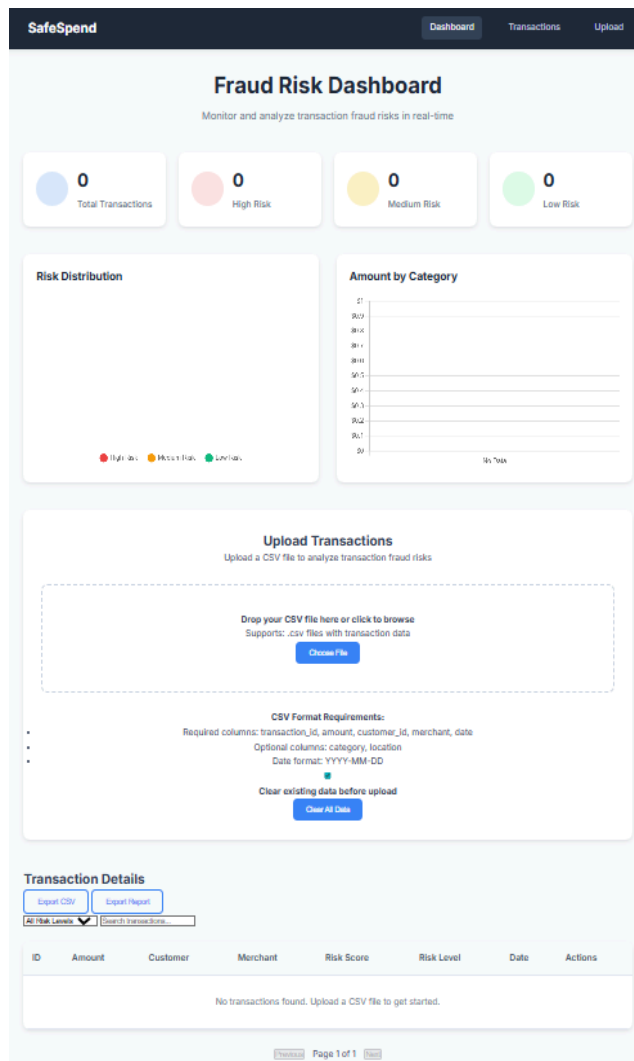
Final Outcome

By performing thorough testing, ensuring that SafeSpend functions correctly and is ready for submission. The testing process helped confirm:

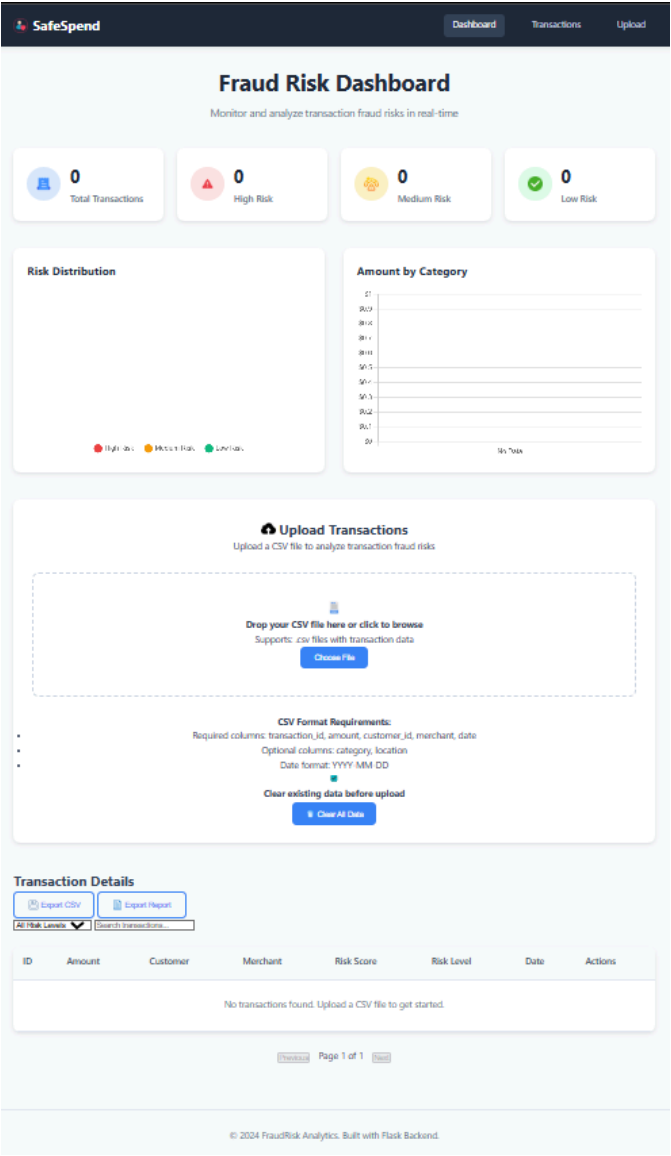
- Transactions display correctly
- Fraud risk labels are consistent
- APIs return valid and correct responses
- Export reports (PDF/CSV) generate properly
- UI is stable and user-friendly

9. Screenshots

1. Dashboard Home Page



This was when there were no icons and no charts dependency

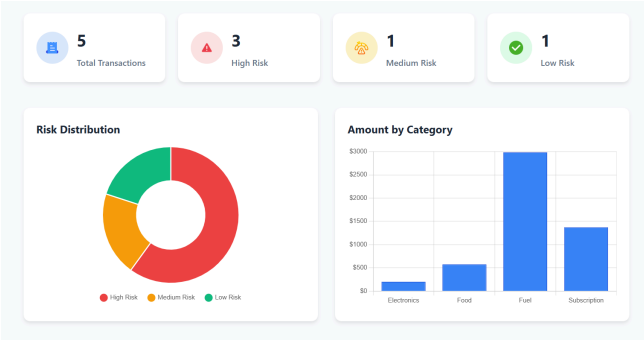
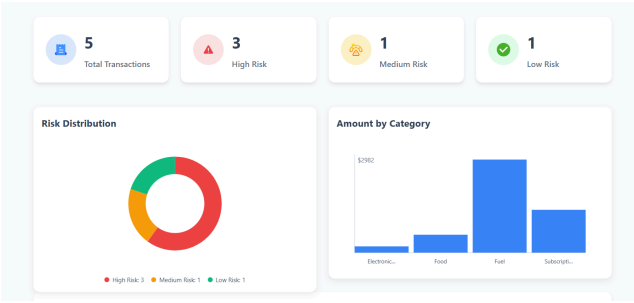


This is with icons And charts dependency

2. Transactions Table

ID	Amount	Customer	Merchant	Risk Score	Risk Level	Date	Actions
TX001	\$200.75	CUST100	Walmart	0.100	low	12/4/2024	View Details
TX002	\$1168.17	CUST101	Subway	0.800	high	19/12/2024	View Details
TX003	\$1813.51	CUST102	Subway	0.800	high	8/8/2024	View Details
TX004	\$1371.43	CUST103	Walmart	0.800	high	18/4/2024	View Details
TX005	\$573.87	CUST104	Whole Foods	0.500	medium	15/6/2024	View Details

3. Fraud Risk Chart



With and without charts dependency

4. Export PDF Report Output

Fraud Risk Report

ID	Amount	Customer	Risk Score	Category
TX001	\$150.50	CUST001	0.10	low
TX002	\$25.75	CUST002	0.10	low
TX003	\$1200.00	CUST003	0.80	high
TX004	\$45.00	CUST004	0.10	low
TX001	\$150.50	CUST001	0.10	low
TX002	\$25.75	CUST002	0.10	low
TX003	\$1200.00	CUST003	0.80	high
TX004	\$45.00	CUST004	0.10	low
TX001	\$200.75	CUST100	0.10	low
TX002	\$1168.17	CUST101	0.80	high
TX003	\$1813.51	CUST102	0.80	high
TX004	\$1371.43	CUST103	0.80	high
TX005	\$573.87	CUST104	0.50	medium

5. Export CSV Report Output

	transaction_id	amount	customer_id	merchant	transaction_category	location	fraud_risk_risk_category	processed_at
1	TX001	200.75	CUST100	Walmart	2024-04-1	Electronics: Los Angeles	0.1 low	2026-01-14T15:48:55.705013
2	TX002	1168.17	CUST101	Subway	2024-12-1	Fuel: Online	0.8 high	2026-01-14T15:48:55.705024
3	TX003	1813.51	CUST102	Subway	2024-08-0	Fuel: Houston	0.8 high	2026-01-14T15:48:55.705026
4	TX004	1371.43	CUST103	Walmart	2024-04-1	Subscription: New York	0.8 high	2026-01-14T15:48:55.705028
5	TX005	573.87	CUST104	Whole Foods	2024-06-1	Food: Houston	0.5 medium	2026-01-14T15:48:55.705030

10. Result & Discussion

SafeSpend successfully displays transaction records, detects fraud risk levels, visualizes fraud patterns using charts, and generates exportable fraud reports in PDF and CSV format. This makes fraud analysis faster and more structured compared to manual monitoring.

11. Conclusion

SafeSpend is an effective fraud monitoring dashboard that combines transaction analysis, fraud risk scoring, visualization, and report exporting features. The project demonstrates a practical full-stack implementation that can support fraud investigation and compliance reporting.

12. Github

<https://github.com/ajaya-bk/Fraud-Risk-Dashboard>

What We Learned From This Project.

Jayant Gautam (Frontend Developer):

Through this project, I gained strong practical experience in designing and developing a complete dashboard interface using HTML, CSS, and JavaScript. I learned how to connect the frontend with backend services through API integration and how to dynamically display transaction data on the dashboard. Initially, the frontend was built without using any chart dependency, but the interface looked very basic and did not represent analytics effectively. To enhance the visual quality and user understanding, I decided to integrate Chart.js, which improved the dashboard's appearance and made fraud insights easier to interpret. Overall, this role helped me improve my skills in UI structuring, dynamic rendering, and building an interactive dashboard experience.

Ajaya BK (Backend Developer):

By working on the backend development, I gained hands-on experience in building REST APIs using Flask and managing real-time communication between the frontend and backend. I learned how to integrate and manage an SQLite database for storing, retrieving, and clearing transaction records efficiently. I also gained knowledge in implementing fraud risk scoring logic so that transactions could be categorized into low, medium, and high-risk levels for better fraud monitoring. In addition, I learned how to generate and export reports in PDF and CSV formats, which is an important feature for self-analysing, audits and documentation. This role strengthened my understanding of backend architecture, database handling, and implementing real-world functionalities like reporting.

Nixon Antony (Tester):

As the tester of the project, I learned the importance of validating both frontend and backend functionalities to ensure system reliability. I gained experience in performing functional testing to confirm that transaction data, risk levels, and dashboard features

worked as expected. I also tested backend APIs to verify correct responses, handled edge cases such as empty data or invalid inputs, and ensured that the system remained stable under different conditions. Additionally, I tested the PDF and CSV export outputs to confirm that reports were generated correctly and contained accurate data. This role improved my skills in manual testing, bug identification, and verifying fixes to deliver a stable final project.